Phase 1: Automated Quality & Governance Evaluation Step

Before deploying any model package to production, a governance check must verify that the model meets performance thresholds (e.g., $\text{PR-AUC} \geq 0.80$ or $\text{ROC-AUC} \geq 0.82$).

Step 1: Create Evaluation Script (`src/evaluate.py`)

Create `src/evaluate.py` to calculate metrics on test data extracted from the offline feature store and generate a standard SageMaker evaluation report JSON.

Python

```python
import os
import json
import tarfile
import pathlib
import pandas as pd
import numpy as np
import xgboost as xgb
from sklearn.metrics import roc_auc_score, precision_recall_curve, auc

def evaluate_model():
    print("--- Governance Evaluation: Testing Model Quality ---")

    # 1. Paths set by SageMaker Pipelines environment
    model_path = "/opt/ml/processing/model/model.tar.gz"
    test_path = "/opt/ml/processing/test/test.csv"
    output_dir = "/opt/ml/processing/evaluation"
    pathlib.Path(output_dir).mkdir(parents=True, exist_ok=True)

    # 2. Extract trained model binary
    with tarfile.open(model_path) as tar:
        tar.extractall(path="./extracted_model")

    booster = xgb.Booster()
    booster.load_model("./extracted_model/xgboost-model")

    # 3. Load Evaluation Data
    df_test = pd.read_csv(test_path)
    X_test = df_test.drop(columns=["Churn"]).values
    y_test = df_test["Churn"].values

    # 4. Generate Predictions
    dtest = xgb.DMatrix(X_test)
    probabilities = booster.predict(dtest)
```

```
    # 5. Compute Metrics
    roc_auc = float(roc_auc_score(y_test, probabilities))
    precision, recall, _ = precision_recall_curve(y_test, probabilities)
    pr_auc = float(auc(recall, precision))

    print(f"Evaluation Complete. ROC-AUC: {roc_auc:.4f} | PR-AUC: {pr_auc:.4f}")

    # 6. Format Standard SageMaker Evaluation Report
    report_dict = {
        "classification_metrics": {
            "roc_auc": {"value": roc_auc, "standard_deviation": "NaN"},
            "pr_auc": {"value": pr_auc, "standard_deviation": "NaN"}
        }
    }

    evaluation_path = os.path.join(output_dir, "evaluation.json")
    with open(evaluation_path, "w") as f:
        f.dump(report_dict, f, indent=4)

    print(f"Saved evaluation report to: {evaluation_path}")

if __name__ == "__main__":
    evaluate_model()
```

Phase 2: Orchestrating the End-to-End CD Pipeline

Step 2: Create SageMaker CI/CD Pipeline (`pipelines/churn_cd_pipeline.py`)

This script defines an automated directed acyclic graph (DAG) using the `sagemaker.workflow` SDK. It ties together data fetching, evaluation, model registration, conditional metrics validation, and deployment.

Python

```
import boto3
import sagemaker
from sagemaker.workflow.pipeline import Pipeline
from sagemaker.workflow.steps import ProcessingStep, CreateModelStep
from sagemaker.workflow.condition_step import ConditionStep
from sagemaker.workflow.conditions import ConditionGreaterThanOrEqualTo
from sagemaker.workflow.properties import PropertyFile
from sagemaker.workflow.model_step import ModelStep
from sagemaker.processing import ScriptProcessor, ProcessingInput, ProcessingOutput
from sagemaker.model import Model
from sagemaker.inputs import CreateModelInput

# Configuration
```

```
REGION = "us-east-1"
BUCKET = "company-mlops-churn-production-1066795149900"
ROLE = sagemaker.get_execution_role()
MODEL_PACKAGE_GROUP_NAME = "customer-churn-model-group"

sagemaker_session = sagemaker.Session()

def build_cd_pipeline():
    # 1. Processing Step: Governance & Metric Evaluation
    image_uri = sagemaker.image_uris.retrieve(framework="sklearn", region=REGION, version="1

    script_eval = ScriptProcessor(
        image_uri=image_uri,
        command=["python3"],
        instance_type="ml.m5.xlarge",
        instance_count=1,
        role=ROLE,
        sagemaker_session=sagemaker_session
    )

    evaluation_report = PropertyFile(
        name="EvaluationReport",
        output_name="evaluation",
        path="evaluation.json"
    )

    step_eval = ProcessingStep(
        name="EvaluateChurnModelQuality",
        processor=script_eval,
        inputs=[
            ProcessingInput(
                source=f"s3://{BUCKET}/models/xgboost-model.tar.gz",
                destination="/opt/ml/processing/model"
            ),
            ProcessingInput(
                source=f"s3://{BUCKET}/processed/customer_churn_processed.csv",
                destination="/opt/ml/processing/test"
            )
        ],
        outputs=[
            ProcessingOutput(output_name="evaluation", source="/opt/ml/processing/evaluation
        ],
        code="src/evaluate.py",
        property_files=[evaluation_report]
    )

    # 2. Condition Step: Governance Quality Gate (PR-AUC >= 0.80)
    cond_gte = ConditionGreaterThanOrEqualTo(
        left=PropertyFileRef(
            step_name=step_eval.name,
            property_file=evaluation_report,
```

```
            json_path="classification_metrics.pr_auc.value"
        ),
        right=0.80
    )

    # 3. Model Registration / Deployment Step (Triggered if Condition succeeds)
    xgb_image_uri = sagemaker.image_uris.retrieve(framework="xgboost", region=REGION, versio

    model = Model(
        image_uri=xgb_image_uri,
        model_data=f"s3://{BUCKET}/models/xgboost-model.tar.gz",
        sagemaker_session=sagemaker_session,
        role=ROLE
    )

    step_create_model = ModelStep(
        name="RegisterApprovedModelPackage",
        step_args=model.register(
            content_types=["text/csv", "application/json"],
            response_types=["text/csv", "application/json"],
            inference_instances=["ml.m5.xlarge"],
            transform_instances=["ml.m5.xlarge"],
            model_package_group_name=MODEL_PACKAGE_GROUP_NAME,
            approval_status="Approved"
        )
    )

    # 4. Construct Pipeline Graph
    pipeline = Pipeline(
        name="CustomerChurn-CD-Pipeline",
        steps=[step_eval, ConditionStep(
            name="CheckQualityGate",
            conditions=[cond_gte],
            if_steps=[step_create_model],
            else_steps=[]
        )],
        sagemaker_session=sagemaker_session
    )

    return pipeline

if __name__ == "__main__":
    pipeline = build_cd_pipeline()
    pipeline.upsert(role_arn=ROLE)
    print("Pipeline compiled and uploaded to SageMaker successfully!")
    execution = pipeline.start()
    print(f"Pipeline execution started. Execution ARN: {execution.arn}")
```

Phase 3: Automated Endpoint Deployment & Provisioning

Once the candidate model passes governance and moves to `Approved` in the Model Registry, the deployment script creates or updates the live SageMaker REST Endpoint.

Step 3: Author Endpoint Deployment Script (`src/deploy_endpoint.py`)

Create `src/deploy_endpoint.py` to deploy the latest approved model package to a production REST endpoint with auto-scaling rules.

Python

```python
import boto3
import time
import sagemaker

sm_client = boto3.client("sagemaker", region_name="us-east-1")
role = sagemaker.get_execution_role()

MODEL_PACKAGE_GROUP = "customer-churn-model-group"
ENDPOINT_CONFIG_NAME = "customer-churn-endpoint-config-prod"
ENDPOINT_NAME = "customer-churn-endpoint-prod"

def deploy_approved_model():
    print(f"--- Step 1: Fetching Latest Approved Model Package from '{MODEL_PACKAGE_GROUP}'

    # 1. Retrieve latest approved model package
    approved_packages = sm_client.list_model_packages(
        ModelPackageGroupName=MODEL_PACKAGE_GROUP,
        ModelApprovalStatus="Approved",
        SortBy="CreationTime",
        SortOrder="Descending"
    )["ModelPackageSummaryList"]

    if not approved_packages:
        raise Exception("No approved model packages found in Model Registry!")

    latest_package_arn = approved_packages[0]["ModelPackageArn"]
    print(f"Found Approved Model Package ARN: {latest_package_arn}")

    # 2. Create SageMaker Model Resource
    model_name = f"churn-model-{int(time.time())}"

    create_model_res = sm_client.create_model(
        ModelName=model_name,
        Containers=[{"ModelPackageName": latest_package_arn}],
        ExecutionRoleArn=role
    )
```

```
    print(f"Created SageMaker Model Resource: {model_name}")

    # 3. Create Endpoint Configuration (Production Compute Setup)
    print("--- Step 2: Provisioning Endpoint Configuration ---")
    sm_client.create_endpoint_config(
        EndpointConfigName=ENDPOINT_CONFIG_NAME,
        ProductionVariants=[
            {
                "VariantName": "AllTraffic",
                "ModelName": model_name,
                "InitialInstanceCount": 1,
                "InstanceType": "ml.m5.xlarge",
                "InitialVariantWeight": 1.0
            }
        ]
    )
    print(f"Endpoint Config '{ENDPOINT_CONFIG_NAME}' created.")

    # 4. Deploy or Update Endpoint
    print(f"--- Step 3: Deploying Endpoint '{ENDPOINT_NAME}' ---")
    try:
        # Check if endpoint already exists
        sm_client.describe_endpoint(EndpointName=ENDPOINT_NAME)
        print(f"Endpoint '{ENDPOINT_NAME}' exists. Performing zero-downtime update...")
        sm_client.update_endpoint(
            EndpointName=ENDPOINT_NAME,
            EndpointConfigName=ENDPOINT_CONFIG_NAME
        )
    except sm_client.exceptions.ClientError:
        print(f"Creating new production endpoint '{ENDPOINT_NAME}'...")
        sm_client.create_endpoint(
            EndpointName=ENDPOINT_NAME,
            EndpointConfigName=ENDPOINT_CONFIG_NAME
        )

    # 5. Wait for Endpoint Status = InService
    print("Waiting for endpoint provisioning to complete (this may take 5-8 minutes)...")
    waiter = sm_client.get_waiter("endpoint_in_service")
    waiter.wait(EndpointName=ENDPOINT_NAME)
    print(f"SUCCESS! Endpoint '{ENDPOINT_NAME}' is now live and status is InService!")

if __name__ == "__main__":
    deploy_approved_model()
```

Phase 4: Configure Auto-Scaling Policies for Endpoint

To fulfill enterprise resilience standards, configure Application Auto Scaling to handle traffic spikes (scaling from 1 to 3 instances automatically).

Step 4: Add Auto-Scaling Script (`src/setup_autoscaling.py`)

Python

```python
import boto3

autoscaling_client = boto3.client("application-autoscaling", region_name="us-east-1")

ENDPOINT_NAME = "customer-churn-endpoint-prod"
RESOURCE_ID = f"endpoint/{ENDPOINT_NAME}/variant/AllTraffic"

def configure_autoscaling():
    print(f"Configuring Auto-Scaling for Endpoint Variant: {RESOURCE_ID}")

    # 1. Register Scalable Target (1 min, 3 max replicas)
    autoscaling_client.register_scalable_target(
        ServiceNamespace="sagemaker",
        ResourceId=RESOURCE_ID,
        ScalableDimension="sagemaker:variant:DesiredInstanceCount",
        MinCapacity=1,
        MaxCapacity=3
    )

    # 2. Define Target Tracking Policy based on Average Invocations Per Minute
    autoscaling_client.put_scaling_policy(
        PolicyName="SageMakerVariantInvocationsPerMinTarget",
        ServiceNamespace="sagemaker",
        ResourceId=RESOURCE_ID,
        ScalableDimension="sagemaker:variant:DesiredInstanceCount",
        PolicyType="TargetTrackingScaling",
        TargetTrackingScalingPolicyConfiguration={
            "TargetValue": 1000.0, # Scale up when invocations exceed 1000/min
            "PredefinedMetricSpecification": {
                "PredefinedMetricType": "SageMakerVariantInvocationsPerInstance"
            },
            "ScaleInCooldown": 300,
            "ScaleOutCooldown": 60
        }
    )
    print("Auto-scaling policy successfully attached!")

if __name__ == "__main__":
    configure_autoscaling()
```

Phase 5: Real-Time REST Inference Verification

Step 5: Test Live Prediction Script (`src/predict.py`)

Verify that the active production endpoint returns live probabilities against an un-headered test payload.

Python

```python
import boto3
import json

runtime_client = boto3.client("sagemaker-runtime", region_name="us-east-1")
ENDPOINT_NAME = "customer-churn-endpoint-prod"

# Sample Customer Feature Vector (33 numerical features corresponding to clean matrix schema
sample_customer_payload = "0,1,0,2,1,1,55.3,110.6,0,0,1,1,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1,

def invoke_inference():
    print(f"Sending REST prediction request to endpoint '{ENDPOINT_NAME}'...")

    response = runtime_client.invoke_endpoint(
        EndpointName=ENDPOINT_NAME,
        ContentType="text/csv",
        Body=sample_customer_payload
    )

    result = response["Body"].read().decode("utf-8")
    churn_probability = float(result.strip())

    print("\n================ PREDICTION RESULT ================")
    print(f"Customer Churn Risk Probability: {churn_probability * 100:.2f}%")
    print(f"Raw Output: {result}")
    print("==================================================\n")

if __name__ == "__main__":
    invoke_inference()
```

Phase 6: Execution Order & Deliverable Verification

Run the components sequentially in your Command Prompt to validate **Deliverable 3**:

DOS

```
# 1. Compile & Execute SageMaker CD Pipeline
python pipelines/churn_cd_pipeline.py
```

```
# 2. Deploy Approved Model Package to Production REST Endpoint
python src/deploy_endpoint.py

# 3. Attach Auto-scaling Guardrails
python src/setup_autoscaling.py

# 4. Verify Real-time REST Inference
python src/predict.py
```